

FPGA Implementation of Coherent Multicore Processor

Roshan Sundaram
roshan.sundaram@duke.edu

March 25, 2026

1 Introduction

Given that ECE 652 is a course focused on multicore architectures and the structures that they are built upon, this project will implement the topics discussed in class in hardware. In ECE 350 (digital design), we built a 5-stage pipelined CPU in Verilog that implemented a modified version of the MIPS architecture. Taking inspiration from that endeavor, this project will focus on integrating multiple CPU cores in Verilog and tying them together with a simple but functional cache coherence policy. The multicore processor will then be synthesized and implemented on an FPGA.

2 Background

Multicore processors make up the vast majority of modern chip designs. However, connecting multiple cores together and exploiting thread-level parallelism relies on the fundamental assumption that the memory system will behave in a predictable, predefined manner. This requirement is known as memory consistency, and is concerned with the global ordering of load and store instructions. Consistency, being an architectural specification, is visible to the programmer. *Micro*architects may implement various microarchitectural features to guarantee consistency, however. To that end, cache coherence protocols are systems that support consistency models but are invisible to the programmer. Various coherence protocols – or no coherence at all – may be used to uphold a consistency model. Within cache coherence, two popular solutions exist: snooping protocols, and directory protocols.

Snooping protocols require cores to broadcast coherence messages regarding the state of a certain cache block to a shared, totally-ordered bus. Each cache monitors the bus and, upon receiving a relevant coherence message from another core, performs the required action on its own cache. Such protocols are conceptually simple since they abstract away the complexities and challenges associated with a “shared bus” but are difficult to scale.

Directory protocols remove some of the overhead of snooping protocols by adding a data structure known as the directory to the last-level cache. For each cache block, the directory tracks which cores have the block, the state of each block in each core, and the owner of the block. When transactions are issued, instead of the message being put onto a shared bus, it is sent directly to the directory, at which point the transaction becomes ordered and is serviced.

The states tracked by a coherence protocol can be as few as two and up to a very large number (if counting transient states). The simplest protocol (applied generally to a snooping or directory protocol) is a valid/invalid protocol, which tracks whether a block is valid or invalid using a validity bit. While such a system is easy to implement, it leads to excessive overhead from coherence transactions. More effective protocols are those which are a subset of the “MOESI” states. In these protocols, each block is said to be in one of five (or fewer) stable states representing the ownership and sharing information of that cache block. When cores read or write data, the protocol dictates state changes.

Connecting multiple cores together in hardware is the job of the interconnection network, which can take on a variety of topologies. Within a physical network, a designer can implement virtual networks, virtual channels, and buffers at the source and destination ends. A robust network is crucial to preventing deadlocks (protocol and routing) and ensuring smooth operation of the multicore processor. For a simple multiprocessor, a simple network may suffice: rings and meshes are conceptually simple and can achieve decent performance.

3 Implementation and Execution

3.1 Hardware Description Language

This project will be designed and implemented in a hardware description language (HDL) and synthesized in AMD Vivado following steps similar to those practiced in ECE 350. The basis of the processor will be the WARP-V configurable RISC core developed by Steve Hoover of Redwood EDA [1]. The WARP-V configurator allows for the generation of a pipelined processor (RISC-V or MIPS) in transaction-level Verilog (TL-Verilog) [2]. TL-Verilog is an HDL built upon SystemVerilog that allows for easy design of pipelined CPUs without explicitly declaring latches and wires. TL-Verilog is then translated to SystemVerilog using Redwood's SandPiper tool, which can be accessed online through their MakerChip development tool or run locally.¹ SystemVerilog can then be synthesized in Vivado and deployed to an FPGA.²

3.2 Cores

WARP-V provides a detailed configurator that can generate everything from a simple, single-cycle processor to a deeply-pipelined, superscalar core. For this project, two simple cores will be used, likely with three to five pipeline stages each for understandability. The cores themselves are not generated with any IO capabilities or cache. Thus, it will be a major effort of this project to implement the private caches and perhaps a shared L2 cache. Multi-cycle functional units like multipliers and dividers may also be omitted due to their high chip area.

3.3 Interconnection Network

Given that this project will consist of just two cores to start off, the choice of interconnection network becomes trivial as scaling will not be an issue. Each core must be linked to the other and to the last level cache (LLC). This goal could be achieved via three bidirectional links (C1 to C2, C1 to LLC, and C2 to LLC). This framework could be modified easily to implement either a snooping or directory protocol.

3.4 Coherence Protocol

A snooping protocol will be used to implement cache coherence in this multicore processor, as it is conceptually simple and only requires the addition of state bits to each cache block and the construction of a bus arbiter to handle interactions between the bus and the cores/caches. This module will be constructed in TL-Verilog or SystemVerilog (depending on the capabilities of TL-Verilog) and will contain the bulk of the logic of the project.

An MSI coherence protocol is a robust protocol that will be used in this project. Although a valid/invalid protocol would have been simpler, MSI offers a better balance of complexity and performance, and can be implemented by following the state transition diagram discussed in class. The analysis of transient states will be a challenge but existing research and literature on MSI protocols will be studied and, if needed, a model checker like Murphi can be used to verify the protocol.

4 Experimental Setup

Testing at every step is crucial to a successful project. First, each Verilog component will be tested using testbenches to verify correct behavior. The skills learned in ECE 350 – notably the use of GTKWave – will assist in the debugging effort. The testbenches will emulate surrounding hardware – such as a dummy core to test the arbiter, or a dummy bus to test the L1 cache – to rigorously verify behavior under a variety of scenarios. Once each component is verified, the full system will be built from the various individual components and tested using microbenchmarks. Microbenchmarks will be written in C, compiled down to either RISC-V or MIPS assembly, and run on the multicore processor.³ Once the final system is synthesized and implemented on the FPGA, the integrated logic analyzer (ILA) and debug cores in Vivado will be used to observe the operation of the coherence protocol and compare against expected values. This stage of testing will be the most time-consuming, as the ILA is notoriously slow to synthesize and use.

¹I have requested an educational copy of SandPiper so I can run the software locally; however, if my request is not approved, the online version should suffice.

²The exact model of FPGA is yet to be determined and will depend on the number of logic cells needed.

³I still have not decided between RISC-V and MIPS. The WARP-V configurator supports both, but I am more familiar with MIPS.

5 Conclusion

The goal of this project is less to advance a cutting-edge area of research and more to implement some very interesting topics in hardware and get practice with building real CPUs. A reasonable timeline for this project would be to generate the WARP-V cores, design the bus & caches in Verilog, and synthesize a basic version of the multicore in Vivado over spring break. Then, during the remainder of March, the custom bus arbiter will be designed in Verilog to implement the MSI protocol. Finally, the remaining weeks in April will see extensive testing of the full system and the writing of the final paper.

References

- [1] Hoover, Steve. "WARP-V." *GitHub Repository*. <https://github.com/stevehoover/warp-v>
- [2] "TL-X." *Transaction-Level Design*. <https://www.tl-x.org>

Progress Report

Roshan Sundaram
Coherent Multicore in Verilog
March 25, 2026

What *Wasn't* Done

Despite anticipating “to generate the WARP-V cores, design the bus & caches in Verilog, and synthesize a basic version of the multicore in Vivado over spring break,” none of that actually happened, as I spent most of break fixing a 3D printer and breaking my head over ethernet cables. . . . So, that left me with the remainder of March to work on what I intended to over break.

What Has Been Done

Bus Arbiter: For the past week-and-a-half since break, I have been focused on designing and testing a bus arbiter in Verilog. The custom module I designed interfaces with two cores and implements the logic to decide which core gets the bus. The logic consists of a round-robin arbiter that waits until a core requests the bus and, depending on which core has priority, grants the bus to a core. The finite state machine (FSM) in this module handles request, grant, and stall signals, while routing snoop requests to the opposing core during memory access.

This module was tested with testbenches in Verilog that emulated two cores connected to this central arbiter. The testbench provided signals similar to what a core would send if it wanted to claim the bus. The signals were simulated in three cases: 1) only core 0 wants the bus, 2) only core 1 wants the bus, and 3) both cores want the bus simultaneously. Examination of GTKWave traces for this testbench initially revealed some bugs in my Verilog which, upon being fixed, functioned as expected (i.e. the core that has priority gets to use the bus while the one that does not must wait for the other core to finish before it can use the bus).

Cache: Since the goal of this project is to *integrate* various Verilog modules into a coherent multicore processor, it seems like a good approach to research existing Verilog cache designs and modify them to implement the MSI coherence protocol. One such example of a cache is: <https://github.com/zebmehring/Processor-Cache>. This project looks to be a student project from a different institution a few years back, and includes various cache levels, but no coherence protocol. The goal of this week (and weekend) will be to finalize the choice of cache design and integrate it into the bus arbiter.

What Shall be Done

There is still much to do, and I admit that my progress has been stunted by two exams on 3/25 and 3/26 that I have been tunnel-visioned on. As a result, I will work extra hard this weekend and over the next few weeks to implement my multicore! With that being said, the main tasks left to complete are related to integrating the cores with the caches. Since the WARP-V cores themselves do not contain traditional data caches, I will need to add my own and make them coherent. I will then need to design a Verilog top module that integrates the bus arbiter, system-level cache, and two cores, and design extensive testbenches for them. The cores will be in RISC-V and, as a result, I will need to use a compiler (or an online tool like Compiler Explorer <https://godbolt.org>). I have not yet thought about testing or benchmarking. . . the task at hand is predominantly to get it *working* before anything else.